Python DSA Roadmap

Complete guide from beginner to advanced — all topics in order

Phase 1 | Python Foundations for DSA

Week 1-2

Built-in Data Types
list, tuple, set, dict — know them deeply

Big O Notation
O(1), O(n), O(n2), O(log n) — measure speed

Python Tricks
Slicing, zip, enumerate, comprehensions

Recursion Basics
Base case, call stack, simple examples

Tip: Start here even if you know Python — DSA uses specific Python patterns you need to master first.

Phase 2 | Linear Data Structures

Week 3-5

Arrays
Indexing, traversal, two-pointer technique

Linked Lists
Singly, doubly, circular — implement from scratch

Stacks
LIFO, use cases: brackets, undo, backtracking

Queues
FIFO, deque, circular queue, priority queue

Hashing
Hash maps, collision handling, frequency count

Strings
Patterns, anagrams, palindromes, sliding window

Tip: Practice 3-5 LeetCode Easy problems per topic. Focus on arrays and hashing first — they appear everywhere.

Phase 3 | Core Algorithms

Week 6-9

Sorting
Bubble, merge, quick, heap sort — write each one

Binary Search
Classic, rotated arrays, search conditions

Two Pointers / Sliding Window
Subarray problems, max/min window, fast+slow

Prefix Sums
Range queries, subarray sums, difference arrays

Greedy
Activity selection, intervals, always pick best

Backtracking
Subsets, permutations, N-queens, sudoku

Tip: Sorting and binary search are the backbone of 40%+ of interview problems. Don't rush this phase.

Phase 4 | Trees & Graphs

Week 10-14

Binary Trees
Inorder, preorder, postorder, level order (BFS)

Binary Search Trees
Insert, delete, search, validate BST

Heaps Min-heap, max-heap, Python heapq, top-K problems	**Trie** Insert/search, autocomplete, word problems
Graph Representation Adjacency list, matrix, directed/undirected	**BFS & DFS** Traversal, shortest path, connected components
Shortest Paths Dijkstra, Bellman-Ford, Floyd-Warshall	**Union-Find** DSU, cycle detection, connected components

Tip: Trees and graphs are the hardest phase. Take your time. BFS/DFS alone unlock 50+ LeetCode problems.

Phase 5 | Dynamic Programming & Advanced

Week 15-20

DP Fundamentals Memoization, tabulation, overlapping subproblems	**Classic DP Problems** Knapsack, LCS, LIS, coin change, fibonacci
DP on Trees/Graphs Tree DP, grid DP, path problems	**Segment & Fenwick Trees** Range queries, point updates, efficient sums
Topological Sort Kahn's algorithm, DFS-based, course schedule	**Bit Manipulation** AND/OR/XOR tricks, power of 2, bitmask DP

Tip: DP is the hardest topic. Start with Fibonacci then coin change then knapsack. Don't jump to hard problems too fast.

Key Python Tools for DSA

• collections.defaultdict, Counter, deque	Essential for hashing, frequency counting, and queue operations
• heapq	Built-in min-heap — use for priority queues and top-K problems
• bisect	Binary search on sorted lists — O(log n) insert and search
• functools.lru_cache	Memoization decorator — makes recursive DP clean and fast
• itertools	permutations, combinations, product — useful in backtracking

---

Practice Strategy

Phase 1-2	LeetCode Easy — 3 to 5 problems per topic
Phase 3-4	LeetCode Medium — 2 problems per day
Phase 5	LeetCode Medium/Hard — focus on DP patterns

Daily goal	1-2 problems minimum, consistency beats intensity

Good luck on your DSA journey! Consistency is the key.